

Building Lists with a Loop

Creation, Traversal and Searching

Unit II • Lecture 16



built by a LOOP, not five separate blocks

Recap: what got repetitive last time

Lecture 15's pattern, three times over

malloc a node -> fill data -> set next = NULL -> link the PREVIOUS node to it -> repeat.
Worked fine for 3 nodes. Utterly impractical for 300.

- A loop can do this exact pattern any number of times.
- The only new idea: we need a pointer that always remembers the CURRENT LAST node, so linking never requires re-walking the list.
- That pointer is usually called tail (or last) — today's genuinely new character.

Algorithm: CREATE a list of n nodes

- 1 Set head = NULL, tail = NULL.
- 2 Repeat n times:
 - 2a Allocate a new node; check for NULL.
 - 2b Fill its data; set its next = NULL.
 - 2c If head == NULL, this is the FIRST node: set head = node.
 - 2d Else: attach after the last node: tail->next = node.
 - 2e Set tail = node (this node is now the last one).
- 3 Return head.

Why the FIRST node needs special treatment

Every other node

tail is a real node.

tail->next = newNode links it in cleanly.

The FIRST node

tail is still NULL — there is no node to attach to yet!

tail->next would crash (NULL has no ->next).

- The fix: check if (head == NULL) first — if so, this new node BECOMES head (and tail).
- Every later node skips that branch and safely uses tail->next.
- This is the loop version of exactly what Lecture 15 did by hand: node 1 was special, nodes 2 and 3 followed the same rule.

createList() — the algorithm in C

```
struct Node *createList(int n)
{
    struct Node *head = NULL;
    struct Node *tail = NULL;

    for (int i = 1; i <= n; i++)
    {
        struct Node *node = malloc(sizeof(struct Node));
        if (node == NULL) exit(1);

        node->data = i * 10;
        node->next = NULL;

        if (head == NULL)
            head = node;
        else
            tail->next = node;

        tail = node;
    }
    return head;
}
```

- head and tail both start NULL — an empty list.
- Each loop pass is exactly one Lecture-15 block, automated.
- tail = node happens EVERY iteration — including the first, so it is ready for iteration two.

Trace: createList(3) — verified

i	New node	head	tail	List so far
1	10	→ 10	→ 10	10 → NULL
2	20	→ 10	→ 20	10 → 20 → NULL
3	30	→ 10	→ 30	10 → 20 → 30 → NULL

head NEVER changes after $i = 1$. Only tail moves forward, one node at a time — exactly why attaching is $O(1)$.

Traversal, formalized

```
void traverse(struct Node *head)
{
    printf("List: ");
    for (struct Node *p = head;
        p != NULL;
        p = p->next)
        printf("%d -> ", p->data);
    printf("NULL\n");
}
```

- This is the exact loop previewed in Lecture 15, now given its own name and its own function.
- Cost: every node is visited exactly once — $O(n)$.
- Every operation for the rest of this unit (search, insert, delete) is built as "traversal, plus one extra step".

Algorithm: SEARCH for a value

- 1 Set position = 1, and p = head.
- 2 While p is not NULL:
 - 2a If p->data equals the key, return position (FOUND).
 - 2b Otherwise, advance: p = p->next, position = position + 1.
- 3 If the loop ends without returning, the key is not in the list — return -1.

This is traversal PLUS ONE extra step: a comparison, and an early return on success.

search() — code and cost

```
int search(struct Node *head, int key)
{
    int position = 1;

    for (struct Node *p = head;
         p != NULL; p = p->next)
    {
        if (p->data == key)
            return position;
        position++;
    }

    return -1;    /* not found */
}
```

Case	Comparisons	Cost
Best (found at head)	1	$O(1)$
Average (found)	$\sim n/2$	$O(n)$
Worst (not found)	n	$O(n)$

Putting it together: createList + traverse + search

```
struct Node *head = createList(5);
```



Call	Result
<code>traverse(head)</code>	10 -> 20 -> 30 -> 40 -> 50 -> NULL
<code>search(head, 30)</code>	30 found at position 3
<code>search(head, 99)</code>	99 not found

This is the real, compiled output — not invented.

Lecture 15 vs Lecture 16: same idea, different scale

Aspect	Lecture 15 (by hand)	Lecture 16 (by loop)
Code for n nodes	n copy-pasted blocks	One loop, any n
Adding a node	Write new code	Just increase n
Risk of typos	High (repetitive by hand)	Low (one block, reused)
New idea required	malloc + linking	+ a tail pointer

Loops replace repetition

- createList uses a tail pointer so every new node attaches in $O(1)$ — without it, each insertion would need an $O(n)$ walk to find the end.
- The first node is special: `head == NULL` means "set head"; every later node uses `tail->next`.
- traverse is the unit's core pattern: start at head, stop at NULL, step with `p = p->next` — $O(n)$.
- search is traversal plus one comparison, with an early return on success — $O(n)$ worst case, and binary search is NOT possible here (no random access).

EXIT TICKET Modify createList so it reads each value with `scanf` instead of using `i * 10`. What line changes, and what stays exactly the same?